

SEMANA 2

POO aplicada: clases, atributos, constructores y métodos

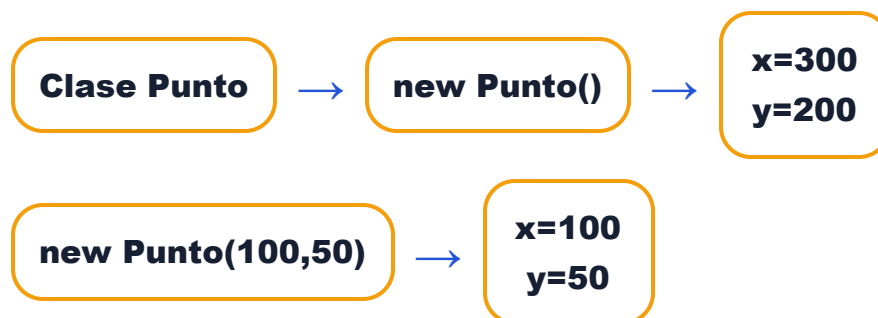
La Semana 2 ya no se trata solo de mover objetos. Ahora importa entender cómo se construye una clase por dentro y cómo cada objeto guarda sus propios datos.

Ej. 1 Punto: clase con atributos X e Y

Qué me piden

- Crear la clase `Punto`.
- No crear subclase de `Actor`.
- Agregar atributos `x` e `y`.
- Constructor por defecto: 300,200.
- Constructor con parámetros.
- Getters y setters.

Cómo entenderlo



► [Ver código orientativo](#)

Ej. 2 PruebaPunto: crear objetos

Qué pasa

- p1 usa el constructor por defecto.
- p2 usa el constructor con parámetros.

Resultado esperado:

p1 = (300,200)

p2 = (100,50)

Pregunta típica: ¿cuál es la diferencia entre `new Punto()` y `new Punto(100,50)` ? El primero usa valores por defecto; el segundo usa valores entregados.

Ej. 3 Abeja: atributos y constructores

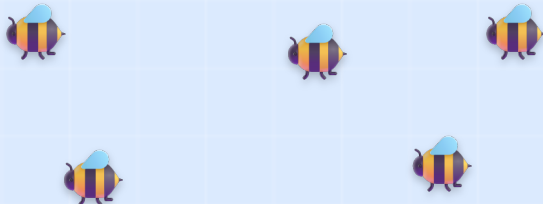
Atributos

- anios
- nombrePanal

Por defecto: 0 años y "Panal 1".

Con parámetros: edad y panal elegidos.

PaisajeEjercicio3



Respuesta: con getters se verifican los datos. Las abejas por defecto quedan con 0 años y Panal 1; las otras pueden tener distintos años y Panal 2.

Ej. 4 Abeja: act(), edad y figura**Qué hace act()****move(200)****turn(-90)****repite**

También se agrega un método para aumentar la edad y otro para mostrar panal + edad.

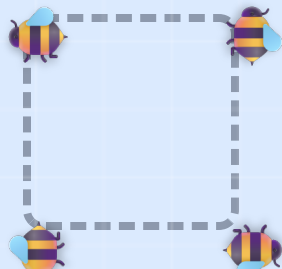
Cuadrado

Figura: cuadrado, porque avanza siempre la misma distancia y gira 90°. $360 \div 90 = 4$ lados.

Ej. 5 Edificio: calcular y analizar**Datos del edificio**

- Cantidad de pisos.
- Departamentos por piso.
- Constructora.
- Si es habitable.

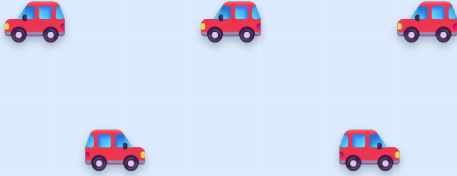
Qué debe hacer analizar()

Retornar un mensaje con departamentos por piso, pisos y total de departamentos.

Total = pisos × departamentos por piso.

Ej. 6 Carro: estado y consumo

Carros



Valores por defecto

- Gasolina: 10 litros.
- Marca: "Sin marca".
- Categoría: C.

Cada avance mueve 20 lugares y resta 0.1 litros.

Respuesta: un carro que se movió no tendrá la misma gasolina que uno que no se movió. El que avanzó consumió combustible.

Si se reinicia el mundo: los objetos vuelven a los valores iniciales definidos por el constructor y por el mundo guardado.

Ej. 7 Cohete: avance + giro

Cohete 1

Avance = 200

Giro = 60°

Forma un hexágono: $360 \div 60 = 6$.

Hexágono



Cohete 2: si el giro es -108° , gira en sentido contrario. El signo negativo indica giro antihorario.

Ej. 8 Meteorito y cohete con giro 120°

Meteorito

No necesita atributos. Solo métodos: avanzar, girar 180° y retroceder.

avanza



gira 180°



retrocede

Cohete con giro 120°

Forma un triángulo: $360 \div 120 = 3$.

Triángulo



Ej. 9 Estrella: giro y contador

Atributos

- `gradosGiro`
- `cantidadMovimientos`

Cada vez que se ejecuta `cambiar()`, la estrella gira y el contador aumenta en 1.

Mar



Respuesta: al presionar Act varias veces, aumenta la cantidad de movimientos realizados y cambia la orientación de la estrella.

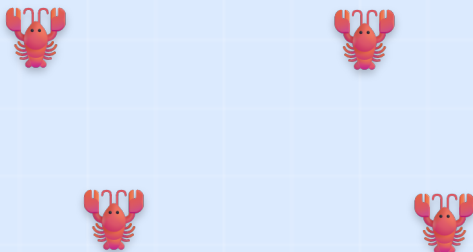
Ej. 10 Langosta: movimiento X e Y

Qué hace

El objeto cambia de posición sumando movimientoX a X y movimientoY a Y.

```
setLocation(getX() + movimientoX,  
            getY() + movimientoY);
```

Movimiento X/Y



Respuesta: al presionar Run, todas las langostas se desplazan continuamente según sus valores de movimientoX y movimientoY. Los positivos y negativos cambian la dirección.

PREGUNTAS TÍPICAS

Relación entre giro y figura geométrica

Cuando un objeto avanza siempre la misma distancia y luego gira siempre el mismo ángulo, puede formar una figura cerrada. La idea clave es dividir 360° por el ángulo de giro.

Giro 90°

$$360 \div 90 = 4$$



Forma un cuadrado.

Giro 60°

$$360 \div 60 = 6$$



Forma un hexágono.

Giro 120°

$$360 \div 120 = 3$$



Forma un triángulo.

CHULETA FINAL

Resumen rápido antes de la prueba

Concepto	Respuesta corta	Ejemplo
Clase	Plantilla para crear objetos.	Abeja, Carro, Punto.
Objeto / instancia	Elemento creado desde una clase.	Una abeja en el mundo.
Atributo	Dato que guarda el objeto.	gasolina, edad, X, Y.
Constructor	Define valores iniciales.	<code>new Punto()</code>
Getter / accesor	Obtiene un dato.	<code>getX()</code> , <code>getPanal()</code>
Setter / mutador	Modifica un dato.	<code>setX(100)</code> , <code>setPanal("Panal 2")</code>
<code>move()</code>	Avanza o retrocede según dirección.	<code>move(20)</code> , <code>move(-20)</code>
<code>turn()</code>	Gira el actor.	<code>turn(45)</code> , <code>turn(-90)</code>
<code>setLocation()</code>	Cambia la ubicación por coordenadas.	<code>setLocation(100,300)</code>
Act	Ejecuta un paso.	Una llamada a <code>act()</code> .
Run	Ejecuta continuamente.	Repite <code>act()</code> .
Reset	Reinicia el mundo.	Vuelve al estado guardado/inicial.

No confundas: “guardar proyecto” no es lo mismo que “guardar mundo”. Guardar proyecto guarda archivos/código; guardar mundo conserva los objetos ubicados en el escenario.

Guía visual de apoyo · POO con Greenfoot · Semanas 1 y 2